

Date: 25/03/2026
Name: Dhanya S

I am Dhanya S from Kakunje Software working as an Intern. Today I had my 43 session it was an interesting session.
Today I created a web page named FoodApp by connecting the frontend and backend that is react and nodejs, checking that in the postman and storing the databases in the MongoDB Compass, and also displaying it in the browser.

MERNSTACK1

1. backend

config

db.js

```
import mongoose from "mongoose";

const connectDB = async () => {

  try {

    await mongoose.connect(process.env.MONGO_URI);

    console.log("MongoDB Connected");

  } catch (error) {

    console.log(error);

    process.exit(1);

  }

};

export default connectDB;
```

controllers

customerController.js

```
import Customer from "../models/Customer.js";

export const createCustomer = async (req, res) => {

  try {

    const customer = await Customer.create(req.body);

    res.status(201).json(customer);

  } catch (error) {

    res.status(500).json({ message: error.message });

  }

};

export const getCustomers = async (req, res) => {

  try {
```

```
    const customers = await Customer.find();
    res.json(customers);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const getCustomerById = async (req, res) => {
  try {
    const customer = await Customer.findById(req.params.id);
    if (!customer) {
      return res.status(404).json({ message: "Customer not found" });
    }
    res.json(customer);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const updateCustomer = async (req, res) => {
  try {
    const customer = await Customer.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    if (!customer) {
      return res.status(404).json({ message: "Customer not found" });
    }
    res.json(customer);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

```

    }
  };

export const deleteCustomer = async (req, res) => {
  try {
    const customer = await Customer.findByIdAndDelete(req.params.id);
    if (!customer) {
      return res.status(404).json({ message: "Customer not found" });
    }
    res.json({ message: "Customer deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

```

foodController.js

```

import Food from "../models/Food.js";

export const createFood = async (req, res) => {
  try {
    const food = await Food.create(req.body);
    res.status(201).json(food);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const getFoods = async (req, res) => {
  try {
    const foods = await Food.find();
    res.json(foods);
  } catch (error) {

```

```

    res.status(500).json({ message: error.message });
  }
};

export const updateFood = async (req, res) => {
  try {
    const food = await Food.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );
    if (!food) {
      return res.status(404).json({ message: "Food not found" });
    }
    res.json(food);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const deleteFood = async (req, res) => {
  try {
    const food = await Food.findByIdAndDelete(req.params.id);
    if (!food) {
      return res.status(404).json({ message: "Food not found" });
    }
    res.json({ message: "Food deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

```

orderController.js

```
import Order from "../models/Order.js";
import Food from "../models/Food.js";

export const createOrder = async (req, res) => {
  try {
    const { customerId, foodId, quantity } = req.body;
    if (!customerId || !foodId || !quantity) {
      return res.status(400).json({ message: "All fields are required" });
    }
    const food = await Food.findById(foodId);
    if (!food) {
      return res.status(404).json({ message: "Food not found" });
    }
    const totalPrice = food.price * quantity;
    const order = await Order.create({
      customerId,
      foodId,
      quantity,
      totalPrice,
      status: "pending"
    });
    res.status(201).json(order);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const getOrders = async (req, res) => {
  try {
```

```
const orders = await Order.find()

  .populate("customerId", "name email")
  .populate("foodId", "name price");
res.json(orders);
} catch (error) {
  res.status(500).json({ message: error.message });
}
};
```

```
export const getOrderById = async (req, res) => {
  try {
    const order = await Order.findById(req.params.id)
      .populate("customerId", "name email")
      .populate("foodId", "name price");
    if (!order) {
      return res.status(404).json({ message: "Order not found" });
    }
    res.json(order);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

```
export const updateOrder = async (req, res) => {
  try {
    const { status } = req.body;
    const validStatus = ["pending", "preparing", "delivered"];
    if (!validStatus.includes(status)) {
      return res.status(400).json({ message: "Invalid status value" });
    }
  }
};
```

```

const order = await Order.findByIdAndUpdate(
  req.params.id,
  { status },
  { new: true }
);
if (!order) {
  return res.status(404).json({ message: "Order not found" });
}
res.json(order);
} catch (error) {
  res.status(500).json({ message: error.message });
}
};

```

```

export const deleteOrder = async (req, res) => {
  try {
    const order = await Order.findByIdAndDelete(req.params.id);
    if (!order) {
      return res.status(404).json({ message: "Order not found" });
    }
    res.json({ message: "Order deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

```

Middleware

authMiddleware.js

```

import jwt from "jsonwebtoken";

```



```
const authMiddleware = (req, res, next) => {  
  try {  
    const token = req.headers.authorization;  
    if (!token) {  
      return res.status(401).json({ message: "No token" });  
    }  
    const decoded = jwt.verify(token, "secretkey");  
    req.user = decoded;  
    next();  
  } catch (error) {  
    res.status(401).json({ message: "Invalid token" });  
  }  
};
```

```
export default authMiddleware;
```

models

Customer.js

```
import mongoose from "mongoose";  
const customerSchema = new mongoose.Schema({  
  name: String,  
  email: String,  
  phone: String,  
  address: String  
}, { timestamps: true });  
export default mongoose.model("Customer", customerSchema);
```

Food.js

```
import mongoose from "mongoose";  
const foodSchema = new mongoose.Schema({
```

```
name: String,
price: Number,
category: String,
isAvailable: {
  type: Boolean,
  default: true
}
}, { timestamps: true });
export default mongoose.model("Food", foodSchema);
```

Order.js

```
import mongoose from "mongoose";
const orderSchema = new mongoose.Schema({
  customerId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Customer"
  },
  foodId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Food"
  },
  quantity: Number,
  totalPrice: Number,
  status: {
    type: String,
    enum: ["pending", "preparing", "delivered"],
    default: "pending"
  }
}, { timestamps: true });
export default mongoose.model("Order", orderSchema);
```

routes

customerRoutes.js

```
import express from "express";  
  
import { createCustomer, getCustomers, getCustomerById, updateCustomer, deleteCustomer  
} from "../controllers/customerController.js";
```

```
const router = express.Router();
```

```
router.post("/", createCustomer);  
router.get("/", getCustomers);  
router.get("/:id", getCustomerById);  
router.put("/:id", updateCustomer);  
router.delete("/:id", deleteCustomer);  
export default router;
```

foodRoutes.js

```
import express from "express";  
  
import { createFood, getFoods, updateFood, deleteFood } from  
"../controllers/foodController.js";
```

```
const router = express.Router();
```

```
router.post("/", createFood);  
router.get("/", getFoods);  
router.put("/:id", updateFood);  
router.delete("/:id", deleteFood);  
export default router;
```

orderRoutes.js

```
import express from "express";
```

```
import { createOrder, getOrders, getOrderById, updateOrder, deleteOrder } from
"./controllers/orderController.js";
```

```
import authMiddleware from "../middleware/authMiddleware.js";
```

```
const router = express.Router();
```

```
router.post("/",authMiddleware, createOrder);
```

```
router.get("/",authMiddleware, getOrders);
```

```
router.get("/:id",authMiddleware, getOrderById);
```

```
router.put("/:id",authMiddleware, updateOrder);
```

```
router.delete("/:id",authMiddleware, deleteOrder);
```

```
export default router;
```

.env

```
MONGO_URI = mongodb://localhost:27017/collection
```

```
JWT_SECRET = mysecretkey
```

package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
```

```
"dependencies": {  
  "bcrypt": "^6.0.0",  
  "bcryptjs": "^3.0.3",  
  "cors": "^2.8.6",  
  "dotenv": "^17.3.1",  
  "express": "^5.2.1",  
  "jsonwebtoken": "^9.0.3",  
  "mongoose": "^9.3.2"  
}  
}
```

server.js

```
import express from "express";  
import cors from "cors";  
import dotenv from "dotenv";  
import connectDB from "./config/db.js";  
import customerRoutes from "./routes/customerRoutes.js";  
import foodRoutes from "./routes/foodRoutes.js";  
import orderRoutes from "./routes/orderRoutes.js";  
  
dotenv.config();  
connectDB();  
const app = express();  
app.use(cors());  
app.use(express.json());  
app.use("/api/customers", customerRoutes);  
app.use("/api/foods", foodRoutes);  
app.use("/api/orders", orderRoutes);  
app.listen(5000,()=>{  
  console.log("server is running on port 5000");  
})
```

2. frontend

src

components

Navbar.jsx

```
import { Link } from "react-router-dom";
import "../styles/Navbar.css";

const Navbar = () => {
  return (
    <nav className="navbar">
      <h1>🍔 Food App</h1>
      <div className="nav-links">
        <Link to="/customers">Customers</Link>
        <Link to="/foods">Foods</Link>
        <Link to="/orders">Orders</Link>
      </div>
    </nav>
  );
};

export default Navbar;
```

pages

Customers.jsx

```
import React, { useEffect, useState } from "react";
import {
  getCustomers,
  addCustomer,
  deleteCustomer,
```

```

    updateCustomer
  } from "../services/customerService";
import "../styles/Customers.css";

const Customers = () => {
  const [customers, setCustomers] = useState([]);
  const [editId, setEditId] = useState(null);
  const [form, setForm] = useState({
    name: "",
    email: "",
    phone: "",
    address: ""
  });

  const fetchCustomers = async () => {
    const res = await getCustomers();
    setCustomers(res.data);
  };

  useEffect(() => {
    fetchCustomers();
  }, []);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (editId) {
      await updateCustomer(editId, form);
      setEditId(null);
    } else {
      await addCustomer(form);
    }
  }
}

```

```
}  
setForm({ name: "", email: "", phone: "", address: "" });  
fetchCustomers();  
};
```

```
const handleEdit = (customer) => {  
  setForm({  
    name: customer.name,  
    email: customer.email,  
    phone: customer.phone,  
    address: customer.address  
  });  
  setEditId(customer._id);  
};
```

```
return (  
  <div className="customers-page">  
    <h2>Customers</h2>  
    <form onSubmit={handleSubmit}>  
      <input  
        placeholder="Name"  
        value={form.name}  
        onChange={(e) => setForm({ ...form, name: e.target.value })}  
      />  
  
      <input  
        placeholder="Email"  
        value={form.email}  
        onChange={(e) => setForm({ ...form, email: e.target.value })}  
      />  
    </form>  
  </div>  
)
```



```
<input
  placeholder="Phone"
  value={form.phone}
  onChange={(e) => setForm({ ...form, phone: e.target.value })}
/>
```

```
<input
  placeholder="Address"
  value={form.address}
  onChange={(e) => setForm({ ...form, address: e.target.value })}
/>
```

```
<button>{editId ? "Update" : "Add"}</button>
</form>
```

```
{customers.map((c) => (
  <div key={c._id} className="customer-card">
    <div className="customer-info">
      <p><strong>{c.name}</strong></p>
      <p>{c.email}</p>
      <p>{c.phone}</p>
    </div>
```

```
    <div className="btn-group">
      <button className="edit-btn" onClick={() => handleEdit(c)}>
        Edit
      </button>
```

```
      <button className="delete-btn" onClick={() =>
deleteCustomer(c._id).then(fetchCustomers)}>
```

```

        Delete
      </button>
    </div>
  </div>
  )})
</div>
);
};

```

```
export default Customers;
```

Foods.jsx

```

import React, { useEffect, useState } from "react";
import {
  getFoods,
  addFood,
  deleteFood,
  updateFood
} from "../services/foodService";
import "../styles/Foods.css";

const Foods = () => {
  const [foods, setFoods] = useState([]);
  const [editId, setEditId] = useState(null);
  const [form, setForm] = useState({
    name: "",
    price: "",
    category: ""
  });

```

```
const fetchFoods = async () => {  
  const res = await getFoods();  
  setFoods(res.data);  
};
```

```
useEffect(() => {  
  fetchFoods();  
}, []);
```

```
const handleSubmit = async (e) => {  
  e.preventDefault();  
  if (editId) {  
    await updateFood(editId, form);  
    setEditId(null);  
  } else {  
    await addFood(form);  
  }  
  setForm({ name: "", price: "", category: "" });  
  fetchFoods();  
};
```

```
const handleEdit = (food) => {  
  setForm({  
    name: food.name,  
    price: food.price,  
    category: food.category  
  });  
  setEditId(food._id);  
};
```

```
return (  
  <div className="foods-page">  
    <h2>Foods</h2>  
    <form onSubmit={handleSubmit}>  
      <input  
        placeholder="Name"  
        value={form.name}  
        onChange={(e) =>  
          setForm({ ...form, name: e.target.value })  
        }  
      />  
  
      <input  
        placeholder="Price"  
        value={form.price}  
        onChange={(e) =>  
          setForm({ ...form, price: e.target.value })  
        }  
      />  
  
      <input  
        placeholder="Category"  
        value={form.category}  
        onChange={(e) =>  
          setForm({ ...form, category: e.target.value })  
        }  
      />  
  
      <button>{editId ? "Update" : "Add"}</button>  
    </form>  
  </div>  
)
```

```

    {foods.map((f) => (
      <div key={f._id} className="food-card">
        <div className="food-info">
          <p><strong>{f.name}</strong></p><br/>
          <p>₹{f.price}</p>
        </div>

        <div className="btn-group">
          <button className="edit-btn" onClick={() => handleEdit(f)}>
            Edit
          </button>

          <button className="delete-btn" onClick={() => deleteFood(f._id).then(fetchFoods)}>
            Delete
          </button>
        </div>
      </div>
    ))}
  </div>

);
};

```

```
export default Foods;
```

Orders.jsx

```

import React, { useEffect, useState } from "react";
import {
  getOrders,
  createOrder,

```

```
    deleteOrder,  
    updateOrder  
  } from "../services/orderService";  
import { getCustomers } from "../services/customerService";  
import { getFoods } from "../services/foodService";  
import "../styles/Orders.css";
```

```
const Orders = () => {  
  const [orders, setOrders] = useState([]);  
  const [customers, setCustomers] = useState([]);  
  const [foods, setFoods] = useState([]);  
  const [form, setForm] = useState({  
    customerId: "",  
    foodId: "",  
    quantity: 1  
  });
```

```
  const fetchData = async () => {  
    const ordersRes = await getOrders();  
    const customersRes = await getCustomers();  
    const foodsRes = await getFoods();  
    setOrders(ordersRes.data);  
    setCustomers(customersRes.data);  
    setFoods(foodsRes.data);  
  };
```

```
  useEffect(() => {  
    fetchData();  
  }, []);
```

```

const handleSubmit = async (e) => {
  e.preventDefault();
  await createOrder(form);
  setForm({
    customerId: "",
    foodId: "",
    quantity: 1
  });
  fetchData();
};

```

```

return (
  <div className="orders-page">
    <h2>Orders</h2>
    <form onSubmit={handleSubmit}>
      <select
        value={form.customerId}
        onChange={(e) =>
          setForm({ ...form, customerId: e.target.value })
        }>
        <option value="">Select Customer</option>
        {customers.map((c) => (
          <option key={c._id} value={c._id}>
            {c.name}
          </option>
        ))}
      </select>

      <select
        value={form.foodId}

```

```

    onChange={(e) =>
      setForm({ ...form, foodId: e.target.value })
    }>
    <option value="">Select Food</option>
    {foods.map((f) => (
      <option key={f._id} value={f._id}>
        {f.name}
      </option>
    ))}
  </select>

  <input
    type="number"
    placeholder="Quantity"
    value={form.quantity}
    onChange={(e) => setForm({ ...form, quantity: e.target.value })} />

  <button>Create Order</button>
</form>

```

```

    {orders.map((o) => (
  <div key={o._id} className="order-card">
    <div className="order-info">
      <p><strong>Customer:</strong> {o.customerId?.name}</p>
      <p><strong>Food:</strong> {o.foodId?.name}</p>
      <p><strong>Quantity:</strong> {o.quantity}</p>
      <p><strong>Total:</strong> ₹ {o.totalPrice}</p>
      <p><strong>Status:</strong> {o.status}</p>
    </div>

```



```

    <div className="btn-group">
      <select
        className="status-select"
        value={o.status}
        onChange={(e) =>
          updateOrder(o._id, { status: e.target.value }).then(fetchData)}>

        <option value="pending">pending</option>
        <option value="preparing">preparing</option>
        <option value="delivered">delivered</option>
      </select>

      <button
        className="delete-btn" onClick={() => deleteOrder(o._id).then(fetchData)}>
        Delete
      </button>
    </div>
  </div>
  )}
</div>
);
};

```

```
export default Orders;
```

services

api.jsx

```

import axios from "axios";
const API = axios.create({
  baseURL: "http://localhost:5000/api"

```

```
});  
  
export default API;
```

customerService.jsx

```
import API from "./api";  
  
export const getCustomers = () => API.get("/customers");  
export const addCustomer = (data) => API.post("/customers", data);  
export const deleteCustomer = (id) => API.delete(`/customers/${id}`);  
export const updateCustomer = (id, data) => API.put(`/customers/${id}`, data);
```

foodService.jsx

```
import API from "./api";  
  
export const getFoods = () => API.get("/foods");  
export const addFood = (data) => API.post("/foods", data);  
export const deleteFood = (id) => API.delete(`/foods/${id}`);  
export const updateFood = (id, data) => API.put(`/foods/${id}`, data);
```

orderService.jsx

```
import API from "./api";  
  
export const getOrders = () => API.get("/orders");  
export const createOrder = (data) => API.post("/orders", data);  
export const updateOrder = (id, data) => API.put(`/orders/${id}`, data);  
export const deleteOrder = (id) => API.delete(`/orders/${id}`);
```

styles

Customers.css

```
.customers-page {
```

```
max-width: 800px;
margin: auto;
padding: 20px;
font-family: Arial, sans-serif;
}
.customers-page h2 {
text-align: center;
margin-bottom: 20px;
color: black;
}
.customers-page form {
display: flex;
flex-direction: column;
gap: 12px;
margin-bottom: 30px;
}
.customers-page form input {
padding: 10px;
border: 1px solid #ccc;
border-radius: 6px;
font-size: 14px;
outline: none;
transition: 0.2s;
}
.customers-page form input:focus {
border-color: green;
}
.customers-page form button {
padding: 10px;
background-color: #4CAF50;
```

```
color: white;
border: none;
border-radius: 6px;
cursor: pointer;
transition: 0.3s;
}

.customers-page form button:hover {
  background-color: green;
}

.customers-page .customer-card {
  background: #f5dddd;
  padding: 15px;
  margin-bottom: 15px;
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  display: flex;
  justify-content: space-between;
  align-items: center;
  transition: 0.2s;
}

.customers-page .customer-card:hover {
  transform: scale(1.01);
}

.customers-page .customer-info {
  font-size: 14px;
  color: black;
}

.customers-page .customer-info p {
  margin: 2px 0;
}
```

```
.customers-page .btn-group {  
  display: flex;  
  gap: 8px;  
}  
.customers-page .btn-group button {  
  padding: 6px 12px;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
  transition: 0.3s;  
}  
.customers-page .edit-btn {  
  background-color: #2196F3;  
  color: white;  
}  
.customers-page .edit-btn:hover {  
  background-color: #1976d2;  
}  
.customers-page .delete-btn {  
  background-color: #f44336;  
  color: white;  
}  
.customers-page .delete-btn:hover {  
  background-color: #d32f2f;  
}
```

Foods.css

```
.foods-page {  
  max-width: 800px;  
  margin: auto;
```

```
padding: 20px;
font-family: Arial, sans-serif;
}
.foods-page h2 {
text-align: center;
margin-bottom: 20px;
color: black;
}
.foods-page form {
display: flex;
flex-direction: column;
gap: 12px;
margin-bottom: 30px;
}
.foods-page form input {
padding: 10px;
border: 1px solid #ccc;
border-radius: 6px;
font-size: 14px;
outline: none;
transition: 0.2s;
}
.foods-page form input:focus {
border-color: #ff9800;
}
.foods-page form button {
padding: 10px;
background-color: orange;
color: white;
border: none;
```

```
border-radius: 6px;
cursor: pointer;
font-size: 15px;
transition: 0.3s;
}
.foods-page form button:hover {
  background-color: green;
}
.foods-page .food-card {
  background: #f9e9d7;
  padding: 15px;
  margin-bottom: 15px;
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  display: flex;
  justify-content: space-between;
  align-items: center;
  transition: 0.2s;
}
.foods-page .food-card:hover {
  transform: scale(1.01);
}
.foods-page .food-info {
  font-size: 14px;
  color: black;
}
.foods-page .food-info p {
  margin: 3px 0;
}
.foods-page .btn-group {
```

```
display: flex;
gap: 8px;
}
.foods-page .btn-group button {
padding: 6px 12px;
border: none;
border-radius: 5px;
cursor: pointer;
transition: 0.3s;
}
.foods-page .edit-btn {
background-color: #2196F3;
color: white;
}
.foods-page .edit-btn:hover {
background-color: #1976d2;
}
.foods-page .delete-btn {
background-color: #f44336;
color: white;
}
.foods-page .delete-btn:hover {
background-color: #d32f2f;
}
```

Navbar.css

```
.navbar {
display: flex;
justify-content: space-between;
align-items: center;
```



```
background-color: #333;
padding: 12px 20px;
color: white;
}
.navbar h1 {
font-size: 23px;
margin: 0;
color: rgb(253, 5, 125);
}
.nav-links {
display: flex;
gap: 20px;
}
.nav-links a {
text-decoration: none;
color: white;
font-size: 16px;
transition: 0.3s;
}
.nav-links a:hover {
color: rgb(253, 5, 125);
}
.nav-links a:active {
transform: scale(0.95);
}
```

Orders.css

```
.orders-page {
max-width: 850px;
margin: auto;
```

```
padding: 20px;
font-family: Arial, sans-serif;
}
.orders-page h2 {
text-align: center;
margin-bottom: 20px;
color: black;
}
.orders-page form {
display: flex;
flex-direction: column;
gap: 12px;
margin-bottom: 30px;
}
.orders-page form select,
.orders-page form input {
padding: 10px;
border: 1px solid #ccc;
border-radius: 6px;
font-size: 14px;
outline: none;
transition: 0.2s;
}
.orders-page form select:focus,
.orders-page form input:focus {
border-color: #9c27b0;
}
.orders-page form button {
padding: 10px;
background-color: #9c27b0;
```

```
color: white;
border: none;
border-radius: 6px;
cursor: pointer;
transition: 0.3s;
}
.orders-page form button:hover {
  background-color: #7b1fa2;
}
.orders-page .order-card {
  background: #e6ddfb;
  padding: 15px;
  margin-bottom: 15px;
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  display: flex;
  justify-content: space-between;
  align-items: center;
  transition: 0.2s;
}
.orders-page .order-card:hover {
  transform: scale(1.01);
}
.orders-page .order-info {
  font-size: 14px;
  color: black;
}
.orders-page .order-info p {
  margin: 3px 0;
}
```

```
.orders-page .status-select {  
  padding: 6px;  
  border-radius: 5px;  
  border: 1px solid #ccc;  
}  
  
.orders-page .btn-group {  
  display: flex;  
  gap: 8px;  
  align-items: center;  
}  
  
.orders-page .delete-btn {  
  padding: 6px 12px;  
  border: none;  
  border-radius: 5px;  
  background-color: #f44336;  
  color: white;  
  cursor: pointer;  
  transition: 0.3s;  
}  
  
.orders-page .delete-btn:hover {  
  background-color: #d32f2f;  
}
```

App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";  
import Customers from "./pages/Customers";  
import Foods from "./pages/Foods";  
import Orders from "./pages/Orders";  
import Navbar from "./components/Navbar";
```

```
function App() {  
  return (  
    <BrowserRouter>  
      <Navbar />  
      <Routes>  
        <Route path="/customers" element={<Customers />} />  
        <Route path="/foods" element={<Foods />} />  
        <Route path="/orders" element={<Orders />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

```
export default App;
```

main.jsx

```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import './index.css'  
import App from './App.jsx'  
  
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

package.json

```
{  
  "name": "frontend",
```

```
"private": true,  
"version": "0.0.0",  
"type": "module",  
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint .",  
  "preview": "vite preview"  
},  
"dependencies": {  
  "axios": "^1.13.6",  
  "react": "^19.2.4",  
  "react-dom": "^19.2.4",  
  "react-router-dom": "^7.13.2"  
},  
"devDependencies": {  
  "@eslint/js": "^9.39.4",  
  "@types/react": "^19.2.14",  
  "@types/react-dom": "^19.2.3",  
  "@vitejs/plugin-react": "^6.0.1",  
  "eslint": "^9.39.4",  
  "eslint-plugin-react-hooks": "^7.0.1",  
  "eslint-plugin-react-refresh": "^0.5.2",  
  "globals": "^17.4.0",  
  "vite": "^8.0.1"  
}  
}
```

Customers

Add

dhanya salian
dhanya123@gmail.com
8197570074

Edit

Delete

Customers

Add

dhanya salian
dhanya222@gmail.com
9997570074

Edit

Delete

Type a query: { field: 'value' } or [Generate query](#)

ExplainResetFindOptions

ADD DATAUPDATEDELETEEXPORT DATAEXPORT CODE

251 - 1 of 1

```
{
  "_id": ObjectId('69c3e08ae4b791cd2a95ee94'),
  "name": "dhanya salian",
  "email": "dhanya222@gmail.com",
  "phone": "9997570074",
  "address": "mangalore",
  "createdAt": 2026-03-25T13:18:02.911+00:00,
  "updatedAt": 2026-03-25T13:19:09.372+00:00,
  "__v": 0
}
```

Foods

Add

chicken biryani	EditDelete
₹250	

Foods

Update

chicken biryani

₹250

EditDelete

Foods

Add

chicken biryani

₹200

Edit

Delete

Foods

Add

chicken biryani

₹200

Edit

Delete

veg biryani

₹180

Edit

Delete

Foods

Name
Price
Category

Add

chicken biryani

₹200

Edit

Delete

Type a query: { field: 'value' } or [Generate query](#)

ExplainResetFindOptions

ADD DATA UPDATEDELETEEXPORT DATAEXPORT CODE

251 - 1 of 1

<pre>_id: ObjectId('69c3e11be4b791cd2a95ee9d') name: "chicken biryani" price: 200 category: "non veg" isAvailable: true createdAt: 2026-03-25T13:20:27.367+00:00 updatedAt: 2026-03-25T13:21:12.011+00:00 __v: 0</pre>
--

Orders

Select Customer



Select Food



1

Create Order

Customer: dhanya salian**Food:** chicken biryani**Quantity:** 2**Total:** ₹400**Status:** pending

pending



Delete

Orders

Select Customer



Select Food



1

Create Order

Customer: dhanya salian**Food:** chicken biryani**Quantity:** 2**Total:** ₹400**Status:** preparing

preparing



Delete

Documents 0

Aggregations

Schema

Indexes 1

Validation

⌚

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

Options

ADD DATA

UPDATE

DELETE

EXPORT DATA

EXPORT CODE

25

1 - 1 of 1

↺

↻

↷

☰

{ }

⌂

```
_id: ObjectId('69c3e1a7e4b791cd2a95eeb0')
customerId: ObjectId('69c3e08ae4b791cd2a95ee94')
foodId: ObjectId('69c3e11be4b791cd2a95ee9d')
quantity: 2
totalPrice: 400
status: "preparing"
createdAt: 2026-03-25T13:22:47.186+00:00
updatedAt: 2026-03-25T13:23:04.559+00:00
__v: 0
```